

Rainbow Tables

Your Hash Types guide's Section 8 flagged salting as the thing that makes rainbow tables "useless" without fully explaining what a rainbow table actually *is* or why salting specifically breaks it — this guide is that missing piece. Rainbow tables represent a completely different strategy from everything in your JtR, Hashcat, and Wordlists guides: those are all about generating and hashing candidates **at attack time**; a rainbow table is about doing an enormous amount of hashing work **once, in advance**, then trading that precomputed effort for near-instant lookups later. Worth locking in the distinction immediately: **Part 1 covers how rainbow tables are actually constructed and used** (the reduction-function/chain structure that makes the whole trick possible) — **Part 2 covers why they've become largely obsolete against modern targets, and the tooling/practical considerations when they're still relevant**. Same underlying goal as every other guide in this series — recover a plaintext from a hash — solved via a completely different tradeoff between storage and computation.

1. The Core Tradeoff Rainbow Tables Exploit — The Foundation

Every cracking approach in this series faces the same fundamental tension: brute-forcing every possible plaintext at attack time is slow; storing every possible plaintext-hash pair precomputed would be instant to look up but requires **impossibly large storage** for any realistic password space.

Naive precomputed lookup table:

Store EVERY (plaintext, hash) pair directly → instant lookup, but storage requirement is the FULL SIZE of the entire keyspace - for an 8-character alphanumeric space (~218 trillion combinations, per your JtR guide's Section 5), storing every pair directly is flatly infeasible on any reasonable hardware

Brute-force at attack time (JtR/Hashcat):

Compute EVERY hash fresh, every single attack → minimal storage, but full computational cost paid EVERY time you attack a new hash, even if you've cracked this exact plaintext before

The single fact that makes rainbow tables possible — a middle ground exists between these two extremes: precompute chains of hash-and-reduce

operations instead of every individual pair, storing only the **START** and **END** of each chain, and reconstruct the rest on demand at lookup time. This trades a large but *manageable* amount of storage for a large but *front-loaded, one-time* amount of computation — the expensive part happens once, during table generation, and every subsequent lookup against that table is comparatively cheap.

PART 1: How Rainbow Tables Actually Work

2. The Reduction Function — The Piece That Makes This Possible

A rainbow table doesn't just chain hashes together (`hash(hash(hash(...)))`) — that alone wouldn't help, since you'd still need to store every intermediate value to reverse it. The key addition is a **reduction function**: a deterministic function that maps a hash *back* into something that looks like a plaintext candidate (not a true inverse — just a different plausible-looking string), so the chain can alternate between hash space and plaintext space:

```
H = hash function (e.g. MD5)
R = reduction function (NOT a true inverse - just maps a hash
    digest to SOME plaintext-shaped string, deterministically)

Chain construction:
plaintext_0 --H--> hash_0 --R--> plaintext_1 --H--> hash_1 --R-->
plaintext_2 --H--> hash_2 --R--> ... (repeated N times) --H-->
hash_N

Only plaintext_0 (chain START) and hash_N (chain END) are STORED.
Every intermediate plaintext_1...plaintext_N-1 and hash_0...hash_N-1
is DISCARDED and regenerated on demand later if needed.
```

This is the entire trick — storage cost per chain is just two values (start and end), no matter how long the chain is, while the chain implicitly "covers" every intermediate plaintext along its length.

3. Why "Rainbow" — Different Reduction Functions Per Column

A naive version of Section 2 (using the **same** reduction function at every step) suffers from a serious flaw: different chains frequently **merge** into each other once they hit the same intermediate value, wasting enormous coverage since

merged chains are redundant. Rainbow tables fix this by using a **different reduction function at each position/column** in the chain:

```
Standard chain (single reduction function R):
chain A: plaintext_0 -H-> hash -R-> plaintext_1 -H-> hash -R-> ...
chain B: plaintext_5 -H-> hash -R-> plaintext_1 -H-> hash -R-> ...
        (if chain A and B ever hit the SAME plaintext_1, they
         MERGE and stay merged for the rest of their length -
         pure wasted coverage)

Rainbow chain (R1, R2, R3... a DIFFERENT reduction function per column):
chain A: plaintext_0 -H-> hash -R1-> plaintext_1 -H-> hash -R2-> ...
chain B: plaintext_5 -H-> hash -R1-> plaintext_1 -H-> hash -R2-> ...
        (even if they collide at COLUMN 1, using a DIFFERENT
         reduction function R2 at the NEXT step means they
         diverge again immediately - merges become far less
         costly, since two chains only stay permanently merged
         if they collide at the SAME column with the SAME
         reduction function)
```

This column-specific reduction function is the specific innovation (Philippe Oechslin, 2003) that gives the technique its name and made it practically viable at scale compared to earlier simple hash-chain proposals.

4. The Lookup Process — Reconstructing a Chain on Demand

Given a target hash to crack, the table isn't searched directly — instead, the lookup **re-walks** a hypothetical chain forward from the target hash, checking at each step whether the resulting endpoint matches any STORED chain end:

1. Take the target hash. Apply the LAST reduction function (R_N) to it, producing a candidate plaintext
2. Check: does this candidate match any chain's STORED END VALUE in the table?
3. If NOT: apply H, then the SECOND-TO-LAST reduction function (R_{N-1}), producing another candidate - check again
4. Repeat, working backward through possible chain lengths, until a stored chain end matches
5. Once a matching chain end is found, RECONSTRUCT that entire chain from its STORED START, re-running H and R forward step-by-step, until the target hash itself appears mid-chain
6. The plaintext immediately preceding that hash in the reconstructed chain is your answer

This lookup process requires re-computing a chain's worth of hash/reduce operations per attempt — genuinely slower than a flat hash-table lookup, but still vastly cheaper than the original brute-force computation the table

represents, since you're walking one chain rather than regenerating the entire precomputed keyspace.

5. Building and Using a Table — Tooling

```
rtgen md5 loweralpha-numeric 1 7 0 3800 33554432 0
rtsort *.rt
rcrack *.rt -h 5f4dcc3b5aa765d61d8327deb882cf99
```

```
rtgen params (RainbowCrack project's classic toolset):
md5          → hash algorithm
loweralpha-numeric → charset
1 7          → min/max plaintext length
0            → reduction function index/offset
3800         → chain length
33554432     → number of chains to generate
0            → table index (for splitting
              generation across multiple runs)

rtsort → sorts generated tables by chain-end value, required for efficient lookup
rcrack → performs the actual lookup against a target hash
```

Ophcrack → a well-known GUI-driven tool specifically built around rainbow tables for cracking Windows LM/NTLM hashes, ships with precomputed tables for common Windows password patterns

PART 2: Why Rainbow Tables Have Become Largely Obsolete — And When They Still Matter

6. Salting Defeats Rainbow Tables Structurally, Not Just Practically

Your Hash Types guide's Section 8 stated this as a fact — here's the actual mechanism. A rainbow table is generated for **one specific hash function with no salt** baked into the chain construction. The moment a salt is introduced:

```
Unsalted: hash("password123") → SAME hash, every time, everywhere
          → one precomputed table works against EVERY unsalted
             instance of this hash function, for every target,
             forever
```

```
Salted:    hash("password123" + salt_A) → hash_A
           hash("password123" + salt_B) → hash_B (COMPLETELY
                                           different from hash_A,
                                           despite the identical
                                           plaintext)
           → a table would need to be regenerated PER SALT VALUE
```

If salts are properly random and, say, 16 bytes long:
2¹²⁸ possible salt values
→ generating a separate rainbow table for every possible salt is exactly as infeasible as brute-forcing the original keyspace directly - salting doesn't just inconvenience the attacker, it structurally destroys the entire "precompute once, reuse against every target" value proposition the table was built on

This is why rainbow tables are effectively a **dead technique against any modern, properly-salted password store** (bcrypt, scrypt, Argon2, or even correctly-implemented salted SHA-256 all defeat this approach completely) — not weakened, but structurally inapplicable.

7. Where Rainbow Tables Still Have Real Relevance Today

Legacy unsalted hash formats still encountered in the wild:

- Unsalted MD5/SHA1 in old/poorly-maintained applications
- Windows LM hashes (the pre-NT, extremely weak legacy Windows hash format - split into two 7-character halves, case-insensitive, making the effective keyspace small enough that precomputed tables covering nearly the ENTIRE space are genuinely feasible and were historically very effective)
- NTLM specifically (Section 4 of your Hash Types guide) is ALSO unsalted, meaning rainbow tables remain structurally applicable against it - though in practice, modern GPU-based brute-force/mask attacks (per your Hashcat guide) have largely overtaken rainbow tables even here, since GPU compute has become cheap enough that the storage-for-computation tradeoff rainbow tables offer is less valuable than it was in 2003
- Any custom/homegrown hashing scheme that skips salting - genuinely still found in real engagements against legacy or poorly-designed internal systems

8. Rainbow Tables vs Modern GPU Brute-Force — Why the Tradeoff Shifted

2003 (rainbow tables' original context): GPU compute was not practically available for cracking - trading storage for computation was a GENUINE win, since computation was the scarce/expensive resource

2026 (current context): GPU compute (per your Hashcat guide's benchmark numbers - tens of billions of hashes/second for fast hashes) is now cheap and fast enough that brute-forcing an unsalted MD5/NTLM hash directly, on demand, is often FASTER than the lookup

-and-reconstruct overhead of walking a rainbow table chain (Section 4) - the tradeoff rainbow tables were built around has been largely erased by hardware improvements on the computation side specifically

This is worth understanding as a genuine historical/technical narrative rather than just "rainbow tables are outdated" — the technique wasn't wrong, the relative cost of storage versus computation that it was optimized for has simply shifted dramatically since 2003.

9. Storage Requirements — Why Comprehensive Tables Are Still Large

Even with the chain-compression trick from Section 2, covering a meaningful keyspace still requires substantial storage:

Example: an 8-character alphanumeric+symbol table covering a meaningful fraction of the keyspace commonly runs into TENS to HUNDREDS of gigabytes, even with rainbow-chain compression - genuinely practical to store and distribute, but a real factor in why targeted, smaller tables (Windows LM-specific, common password patterns only) remain more practical than attempting full-keyspace coverage for longer/more complex password spaces

10. Testing / Usage Methodology

1. FIRST confirm the target hash is genuinely UNSALTED (Section 6) - check the hash format per your Hash Types guide's Section 14 identification methodology; if a salt is present in the hash structure, skip this entire technique and go directly to JtR/Hashcat (Sections referenced throughout Parts 1-2 there)
2. Check whether a precomputed table ALREADY EXISTS for this exact hash type/charset/length combination before generating your own - Ophcrack's freely available tables cover common Windows LM/NTLM cases; regenerating from scratch (Section 5) is a genuinely significant time/storage investment
3. If generating your own table, scope the charset/length tightly to your actual target population (per your Wordlists guide's Section 1 "targeted beats generic" principle) - a narrower, well-scoped table is both cheaper to generate and faster to look up against than attempting broad coverage
4. For anything salted (the vast majority of modern targets), rainbow tables are the WRONG tool entirely - redirect to your JtR/Hashcat guides' wordlist/mask/hybrid techniques instead

11. Legal & Ethical Note

Identical authorization requirement to every other tool covered in this series (your JtR, Hashcat, and Wordlists guides' respective notes) - generating or using rainbow tables against hashes you lack explicit authorization to test is illegal in most jurisdictions regardless of technique.

12. Prevention

- SALT EVERY PASSWORD HASH, always, with a unique random value per password (Section 6) - this single measure structurally defeats rainbow tables entirely, independent of which hash algorithm is used underneath
- Use a purpose-built password-hashing function (bcrypt/scrypt/Argon2id, per your Hash Types guide's Section 15) - these build salting into the format by design, removing the risk of a developer forgetting or incorrectly implementing manual salting
- If forced to maintain a legacy unsalted hash format for backward compatibility, treat it as an urgent migration priority rather than a stable state - unsalted hashes remain vulnerable to this ENTIRE technique category regardless of how strong the underlying hash function otherwise is
- Retire LM hash support entirely if any legacy Windows infrastructure still has it enabled (Section 7) - it's been disabled by default in modern Windows for exactly this reason, but can still be found enabled in poorly-maintained legacy environments

13. Practical Lab Example

Ophcrack's demo mode and freely downloadable small tables are the most accessible way to see this entire technique work end-to-end without needing to generate a table yourself first; RainbowCrack's own project documentation walks through rtgen/rtsort/rcrack against a small custom charset, which is the better exercise for actually understanding Section 2-4's chain mechanics rather than just observing a successful crack.

Suggested practice order:

1. Generate a SMALL custom table (short charset, short length) with rtgen against a synthetic unsalted MD5 hash set - deliberately small enough to complete in minutes, to observe the full generate → sort → lookup pipeline firsthand
2. Time the LOOKUP step specifically and compare it against an equivalent JtR/Hashcat brute-force run over the SAME small keyspace (per Section 8's tradeoff discussion) - on a genuinely small keyspace the two approaches should be reasonably close, making the tradeoff concept concrete rather than theoretical
3. Take the SAME target plaintext, hash it WITH a salt this time,

and attempt the identical rainbow table lookup against it - confirm firsthand that it fails completely, making Section 6's "structural, not practical" claim tangible

Kill Chain / ATT&CK / CWE / OWASP — Full Mapping

Framework	Mapping
CWE	CWE-759 (Use of a One-Way Hash without a Salt - the exact CWE this entire technique exploits, first referenced in your Hash Types guide's Section 8), CWE-916 (Use of Password Hash With Insufficient Computational Effort - relevant when the underlying hash is also fast, compounding the unsalted weakness)
OWASP Top 10:2025	A02:2025 Security Misconfiguration / Cryptographic Failures - identical mapping to your Hash Types guide, since missing salt is a cryptographic implementation failure
CVSS	Not directly CVSS-scored (a precomputation technique, not a vulnerability itself) - severity inherits from the missing-salt weakness it exploits and whatever exposed the hash dump initially
Cyber Kill Chain	Actions on Objectives, with a distinct PRIOR Weaponization-adjacent phase unique to this technique - table GENERATION happens well before any specific target is even identified, unlike JtR/Hashcat's attack-time candidate generation
MITRE ATT&CK	T1110.002 (Brute Force: Password Cracking) - same mapping as JtR/Hashcat, since ATT&CK classifies by the resulting technique/objective rather than the specific precomputation strategy used to achieve it

Quick Reference Cheat Sheet

CORE MECHANISM:

Chain: plaintext -H-> hash -R1-> plaintext -H-> hash -R2-> ...
Stored: ONLY chain start + chain end (not intermediate values)
Lookup: work backward from target hash, applying remaining reduction functions, checking for a matching stored chain end, then reconstruct forward to find the plaintext

TOOLING:

rtgen HASH CHARSET MIN MAX ... → generate a table
rtsort *.rt → sort for efficient lookup
rcrack *.rt -h TARGETHASH → perform lookup
ophcrack → GUI tool, ships with Windows LM/NTLM tables

WHEN STILL RELEVANT:

Unsalted MD5/SHA1/SHA256
Windows LM hashes
NTLM (though GPU brute-force
per your Hashcat guide often
now outperforms table lookup)

WHEN STRUCTURALLY DEFEATED:

Any properly salted hash
(bcrypt/scrypt/Argon2/salted
SHA-family) - Section 6's
"regenerate per salt value"
problem makes this the SAME
infeasibility as brute-
forcing the keyspace directly

Root lesson: rainbow tables trade STORAGE for COMPUTATION by precomputing chains instead of individual pairs - a genuinely elegant solution to a problem (computation was scarce) that mostly no longer exists (per Section 8), AND one that salting defeats structurally rather than just practically, which is why "always salt" is the single most repeated defense across this entire guide series.